WILFRID LAURIER UNIVERSITY

Department of Physics and Computer Science

Implementation of the Fischer et al. (2021) Reinforcement Learning Methodology in a Two-Degree-of-Freedom Robotic Arm Training Pipeline

A Technical Implementation Report Prepared for the CP493 Directed Research Project

Prepared by

Ranjot Sandhu

Submitted to

Professor Sukhjit Sehra

Department of Physics and Computer Science

May 6, 2026

Waterloo, Ontario, Canada

Abstract

This report documents the integration of the reinforcement learning training methodology of Fischer, Hoinville, Eickhoff, and Lilienthal (2021), "Reinforcement learning control of a biomechanical model of the upper extremity", Scientific Reports 11:14445, into the existing two-degree-of-freedom planar robotic arm training framework developed under CP493 at Wilfrid Laurier University. The Fischer paper described six methodological pillars: the use of Soft Actor-Critic (SAC) as the primary training algorithm, an adaptive curriculum on the goal-tolerance radius driven by the rolling success rate, a motor-babbling pre-training phase to populate the off-policy replay buffer, two emergent-behaviour validation harnesses (Fitts' Law and the two-thirds Power Law), and Hill-type muscle dynamics on a biomechanically-proportioned seven-degree-of-freedom upper-extremity arm.

All six methodological pillars have been implemented in this work as additive contributions to the existing two-degree-of-freedom training pipeline, distributed across six git commits totalling approximately 2,200 inserted lines of Python source code, accompanied by 79 dedicated smoke tests and verified against the existing 49-test regression suite. The implementation is contained in five new modules and four targeted modifications to existing modules; no existing call site was broken. A unified GUI launcher is provided so that the training and arm-control front ends can be invoked from a single command. The Fitts' Law harness is verified on a synthetic policy that recovers the true regression coefficients to within the discrete-time-step quantisation noise floor, and the two-thirds Power Law harness recovers the analytic slope of negative one third to seven decimal places on a synthetic ellipse. Components requiring MuJoCo musculoskeletal modelling, multi-muscle-per-joint anatomical geometry, and the wiring of the seven-DOF arm into the training environment are explicitly out of scope for this report and are documented as deferred work.

Table of Contents

1. Introduction and Context

1.1 The Source Paper

Fischer, Hoinville, Eickhoff, and Lilienthal (2021) published in Scientific Reports a demonstration that a model-free deep reinforcement learning agent, trained with Soft Actor-Critic on a seven-degree-of-freedom biomechanical model of the human upper extremity implemented in the MuJoCo physics simulator, could learn a goal-reaching policy that reproduced two emergent properties of natural human motor control: the speed-accuracy trade-off described by Fitts' Law (Fitts, 1954) and the two-thirds Power Law of trajectory curvature versus tangential velocity (Lacquaniti, Terzuolo, and Viviani, 1983). The reported coefficient of determination for the Fitts' Law fit was R-squared equals 0.9986, and the Pearson correlation for the Power Law fit was R equals 0.84, both of which approach the values measured for human movements in the same paradigms.

The methodological core of the paper is not the choice of arm model, which is implementation-specific, but the combination of training-algorithm choice, exploration mechanism, and curriculum that allowed the agent to learn the goal-reaching task and the validation instruments that demonstrated the resulting motion was naturalistic. This report documents the integration of those methodological elements into the existing two-degree-of-freedom planar arm training framework already developed under CP493 at Wilfrid Laurier University, thereby establishing both that the framework is capable of supporting the Fischer protocol and that the validation instruments are correct.

1.2 State of the Project Prior to This Work

Prior to this implementation, the project had progressed through eight phases of development. These produced a Gymnasium-compatible custom environment ArmTaskEnv supporting a two-degree-of-freedom planar arm with workspace, goal, and reward configuration; an interactive arm-control graphical user interface with manual joint control, motion recording, and configuration save/load; a training graphical user interface supporting Soft Actor-Critic, Proximal Policy Optimization, and Advantage Actor-Critic algorithms via Stable-Baselines3 with live reward, loss, and entropy plots; a physics-grounded reward function with four scientifically justified penalty terms grounded in classical mechanics, computational neuroscience, and robotics; and a regression test suite covering 49 unit tests in the project_assets/tests directory.

The reward function had been validated on the existing two-degree-of-freedom arm with PPO training runs of approximately 100,000 timesteps, but no implementation of the Fischer protocol had been undertaken. The project at that point cited the Fischer paper in its academic reference report as Section T but did not yet exercise the corresponding methodology in code. Section T positioned the Fischer paper as future work.

1.3 Scope of This Implementation Effort

The work documented in this report covers the integration of all six methodological pillars of the Fischer paper into the existing two-degree-of-freedom training pipeline as additive extensions, plus a unified launcher to simplify operational use. The work was carried out on a dedicated git branch named claude/fischer-integration over six commits, with a pre-work save point at commit f7f90e5 tagged save-point-2026-05-05 and mirrored as the backup branch backup/save-point-2026-05-05 to permit rollback at any point. No existing module was broken; the existing 49-test regression suite continues to pass without modification.

Three components of the Fischer protocol that require either platform-specific dependencies or large-scale environment refactoring are explicitly deferred and are documented in Section 10 of this report. These are: the integration of the seven-degree-of-freedom arm preset into the training environment as the active arm geometry (currently implemented as data only); the construction of an XML musculoskeletal model file for the MuJoCo physics engine; and the modelling of multi-muscle-per-joint anatomical geometry with realistic moment arms. None of these is necessary to demonstrate the methodology of the Fischer paper on the existing two-degree-of-freedom arm.

2. The Fischer 2021 Methodological Pillars

A close reading of the Methods section of Fischer et al. (2021) identifies six discrete methodological elements that together constitute their training and validation protocol. These are listed in Table 1 with the corresponding implementation reference for this work.

#	Pillar	Implementation step	Commit	Status
1	SAC as primary RL algorithm	Step 1: default-algorithm switch and Fischer-aligned hyperparameters	77b7180	Done
2	Adaptive curriculum on goal tolerance	Step 2: AdaptiveCurriculumCallback (60 cm to 2 cm, 80% threshold)	9d31252	Done
3	Motor-babbling pre-training	Step 2: SAC learning_starts = 5000	9d31252	Done
4	Fitts' Law validation	Step 3: FittsLawValidator with closed-form OLS regression	5f99560	Done
5	Two-thirds Power Law validation	Step 4: PowerLawValidator with central-difference 3-point C	5e63467	Done
6	Hill-type muscle dynamics	Step 5: HillTypeMuscle module + 7-DOF Fischer preset	5f37e36	Done

Table 1. *Mapping of Fischer 2021 methodological pillars to implementation steps in this work.*

Pillars one through three relate to training-time setup. Pillars four and five are post-training validation harnesses that quantify whether the resulting policy reproduces the speed-accuracy and trajectory-shape regularities observed in human motor control. Pillar six is the biomechanical actuation law that distinguishes Fischer's setup from a standard kinematic or torque-driven arm. The unified launcher described in Section 8 is not a Fischer pillar but is included in this report because it was added to support practical use of the new capabilities.

2.1 Save Point and Rollback Safety

Before any of the six methodology steps was begun, a save point was created on the main branch capturing the project state including the formal CP493 progress report, the academic reference report, the project handoff document progress.md, and the current trained-model artifacts. This save point is preserved as the immutable git tag save-point-2026-05-05 and as the named branch backup/save-point-2026-05-05, both pointing to commit f7f90e5. At the time of writing, both references remain valid, so the entire Fischer integration can be rolled back in a single git operation if desired.

3. Step 1 — SAC as Primary Algorithm and Configurable Goal Tolerance

The first integration step accomplished two related changes. First, it switched the default training algorithm of the project from Proximal Policy Optimization to Soft Actor-Critic, matching the algorithm choice of the Fischer paper, and added explicit academic citations to the SAC agent's hyperparameter values to document the source of each numerical choice. Second, it generalized the ArmTaskEnv constructor to accept the goal tolerance, orientation tolerance, and hold-velocity tolerance as optional parameters, and added a new method set_goal_tolerance for runtime updates by an external scheduler. These two changes together form the foundation for the curriculum scheduler delivered in Step 2.

3.1 Files Modified

- **src/rl_armMotion/two_d/gui/training_gui.py** — default algorithm string changed from "PPO" to "SAC" in the TrainingGUI constructor signature, the validation fallback, and the command-line argument default.
- **src/rl_armMotion/two_d/models/agents/sac_agent.py** — module docstring and per-hyperparameter inline comments updated to cite Fischer et al. (2021), Sci. Rep. 11:14445, as the source of each value (lr=3e-4, buffer=1M, batch=256, gamma=0.99, tau=0.005, ent_coef=auto).
- **src/rl_armMotion/two_d/environments/task_env.py** — added three DEFAULT_* class constants, three optional constructor parameters (goal_tolerance, orientation_tolerance_deg, hold_velocity_tolerance), and the new method set_goal_tolerance(value) for runtime updates.

3.2 Rationale

Soft Actor-Critic is preferred over Proximal Policy Optimization for continuous-control goal-reaching tasks because of its sample efficiency on continuous action spaces and its native entropy-regularised exploration. Fischer et al. (2021) reported these as essential for learning the seven-degree-of-freedom goal-reaching task on the biomechanical arm. The existing project already supported all three algorithms via Stable-Baselines3, so no new training-loop code was needed; the change is a one-line default.

The configurable-tolerance work was a necessary precondition for the adaptive curriculum delivered in Step 2: that curriculum needs to update the environment's goal tolerance while training is in progress. Before this step the tolerance was hardcoded to ten centimetres in the constructor body. After this step the tolerance is a class-level constant (DEFAULT_GOAL_TOLERANCE = 0.10) used as the constructor default, can be overridden per-instance, and can be changed at any time via set_goal_tolerance. The historical default value of ten centimetres is preserved exactly so no existing test broke.

3.3 Verification

Seven dedicated smoke tests verify (i) that the default constructor preserves the historical ten-centimetre tolerance and ten-degree orientation window, (ii) that the new constructor parameters override the defaults when supplied, (iii) that set_goal_tolerance updates the running value dynamically, (iv) and (v) that set_goal_tolerance rejects zero and negative inputs with ValueError, (vi) that the existing reset and step methods continue to work after a tolerance change, and (vii) that the TrainingGUI default algorithm is now the string SAC. All seven tests pass.

4. Step 2 — Adaptive Curriculum and Motor Babbling

The second integration step delivered two of the six Fischer pillars in a single commit, because they are both training-time mechanisms that integrate naturally with the same Stable-Baselines3 trainer wrapper. The first was the AdaptiveCurriculumCallback class which implements the adaptive goal-tolerance schedule, and the second was the addition of the SAC learning_starts hyperparameter to enable a motor-babbling pre-training phase.

4.1 AdaptiveCurriculumCallback

Fischer et al. (2021) reported that learning a precise goal-reaching policy from the start of training was intractable: the reward landscape is too sparse when the success region is small. Their solution was an adaptive curriculum on the position tolerance. Training begins with a wide goal radius of approximately sixty centimetres so that early random exploration produces frequent successes and the policy receives a strong learning signal. Once the rolling success rate over the recent training window exceeds eighty percent, the tolerance is shrunk multiplicatively, and the cycle repeats until a precision target of approximately two centimetres is reached. The curriculum is described as adaptive because it advances on the agent's performance, not on a fixed timestep schedule.

The implementation in this project is a Stable-Baselines3 BaseCallback located in the new module src/rl_armMotion/two_d/training/curriculum_callback.py. Its constructor accepts six configurable parameters with defaults that match the Fischer protocol exactly: initial_tolerance=0.60 metres, min_tolerance=0.02 metres, success_rate_threshold=0.80, decay_factor=0.80, window_size=50 episodes, and min_episodes_before_decay=20 episodes (the cooldown between consecutive decays which prevents oscillation). All six parameters are validated in the constructor; ten distinct invalid configurations are rejected with ValueError.

4.2 Callback Lifecycle

On training start, the callback applies the initial wide tolerance to the environment by calling set_goal_tolerance(0.60). On every environment step, it inspects the SB3 self.locals["infos"] and self.locals["dones"] arrays to detect completed episodes; for each completed episode it appends one or zero to a deque-based rolling window depending on whether the goal was reached. After updating the rolling window, it tests whether all four decay preconditions are satisfied (the window is full, the cooldown has elapsed, the current tolerance is above the minimum, and the success rate equals or exceeds the threshold) and if so, computes new_tolerance = max(current * decay_factor, min_tolerance), updates the environment, increments the curriculum stage counter, and resets the cooldown. The callback handles three environment topologies: a bare ArmTaskEnv exposing set_goal_tolerance directly, a vectorised environment supporting env_method, and a DummyVecEnv wrapper exposing an .envs attribute through Monitor and TimeLimit wrappers.

4.3 Trainer Integration

The RLTrainerWithMetrics class in src/rl_armMotion/two_d/training/ppo_trainer_wrapper.py was extended with two new constructor parameters: use_curriculum (default None, which auto-resolves to True for SAC and False for PPO and A2C, matching the algorithm scope of the Fischer paper) and curriculum_kwargs (a dict of overrides for the callback constructor). On train(), the trainer instantiates the callback if use_curriculum is true, appends it to the existing GUI callback in a list, and passes the list to model.learn(). The metrics dictionary returned by get_current_metrics() now contains a "curriculum" sub-dictionary exposing current_tolerance, curriculum_stage, recent_success_rate, window_filled, episodes_since_last_decay, and an enabled flag. This allows the GUI metrics panel to surface the curriculum state.

4.4 Motor Babbling via SAC learning_starts

Fischer et al. (2021) initialized their policy by running a motor-babbling phase in which the agent issued uniformly random commands and stored the resulting transitions in the SAC replay buffer before any policy updates began. This populates the buffer with diverse state-action coverage so the off-policy critic has something useful to learn from on its first gradient steps. In Stable-Baselines3 the equivalent mechanism is the learning_starts hyperparameter: the SAC actor acts uniformly at random for the specified number of environment steps and only stores transitions, with policy gradient updates suppressed until the threshold is crossed. The SACAgent.DEFAULT_HYPERPARAMS dictionary was modified to include learning_starts=5000 with an inline comment documenting the equivalence to Fischer's motor-babbling phase. The five-thousand-step default seeds the buffer for the typical training-run length of one hundred thousand to five hundred thousand timesteps without dominating short demonstration runs.

4.5 Verification

Twelve dedicated smoke tests verify (1) that ten distinct invalid constructor configurations are rejected with ValueError, (2) that the default values match the Fischer protocol exactly, (3) that get_progress() handles the empty-window case correctly, (4 and 5) that the decay logic correctly advances stage zero to stage one with tolerance 0.60 to 0.48 metres on a fully-successful window and the cooldown is respected, (6) that the tolerance does not fall below min_tolerance, (7) that a fifty-percent success rate does not advance the stage, (8) that SAC's default hyperparameters now include learning_starts=5000, (9) that the trainer signature accepts use_curriculum and curriculum_kwargs, (10) that the auto-detection logic enables the curriculum for SAC and disables it for PPO with explicit override capability, (11) that the metrics dictionary exposes the curriculum block, and (12) that the existing ArmTaskEnv default tolerance is preserved unchanged. An end-to-end test then runs a brief SAC training session with the curriculum attached and verifies that the curriculum announcement is printed, the rollout completes, and the curriculum-state dictionary is correctly populated.

5. Step 3 — Fitts' Law Validation Harness

Fitts' Law (Fitts, 1954, J. Exp. Psychol. 47:381) is the classical empirical relationship in human motor control between the movement time MT required to reach a target of width W from a starting distance D, and a quantity called the index of difficulty defined as ID = log base 2 of (2D divided by W). The law states that, across a wide range of D and W combinations, MT is a linear function of ID, with slope b determined by the speed of the performing system and intercept a determined by its reaction-time and minimum-movement overhead. This linear relationship has been confirmed for human pointing, mouse cursor control, eye saccades, and primate reaching, with reported coefficients of determination approaching unity. Fischer et al. (2021) reported R-squared = 0.9986 for their trained seven-degree-of-freedom policy on a thirty-six-cell distance-by-width grid.

5.1 Module Layout

The new submodule src/rl_armMotion/two_d/validation contains fitts_law.py with four public types and a package __init__.py that re-exports them. The four types are FittsLawCondition (a single (D, W, ID) cell with input validation), FittsLawTrial (one per-trial outcome with movement time, final distance, and angle), FittsLawResult (the aggregated regression with closed-form OLS coefficients, R-squared, JSON serialisation, and matplotlib plotting), and FittsLawValidator (the sweep driver). The validator constructor type-checks both the model and the environment: the model must expose a .predict(observation, deterministic=...) method, and the environment must expose set_goal_position, set_goal_tolerance, and a dt attribute.

5.2 Sweep Protocol

The validator's run() method takes a configurable distance grid (default six values from 0.20 to 1.20 metres) and a configurable width grid (default six values from 0.02 to 0.30 metres), giving thirty-six combinations by default; for each combination it runs a configurable number of trials (default fifteen) at stratified-then-jittered angles drawn from minus pi over two to plus pi over two; for each trial it resets the environment, places the goal at the chosen distance and angle via the new ArmTaskEnv.set_goal_position method, sets the goal tolerance to the chosen width, and rolls the policy for at most a configurable maximum step count, recording the time to first contact with the W-radius. After all trials complete, the per-condition mean and standard deviation of MT and the per-condition success rate are computed; conditions with zero successful trials are excluded from the regression (their MT is reported as NaN) but are otherwise retained in the result for diagnostic purposes.

5.3 Regression

The regression is closed-form ordinary least squares over the (ID, MT-mean) pairs of all successful conditions. The slope, intercept, and R-squared are computed from the standard centred-moment formulae. If fewer than two valid conditions exist (degree-of-freedom protection), the regression returns NaN for all three coefficients. Single-condition sweeps are handled gracefully without exceptions.

5.4 Environment Hook

The validator required a new method on ArmTaskEnv to place an arbitrary goal in the workspace coordinate frame. The new method set_goal_position(position) accepts a 2D point, validates its shape, sets the internal goal-mode flag to the new value "EXPLICIT", and recomputes the goal axis as the unit vector from the shoulder base position to the goal. The EXPLICIT mode persists across reset, so the validator can run many trials against the same goal placement without re-configuring between resets. This was added in the same commit as the Fitts' Law module so that the harness is self-contained.

5.5 Verification

Fourteen dedicated smoke tests cover the full surface area of the harness, including the index-of-difficulty arithmetic, condition input validation, the EXPLICIT-mode persistence across reset, position-shape validation, validator type checks, a synthetic-policy benchmark that recovers the true regression coefficients to within the dt-quantisation noise floor with R-squared above 0.997, JSON round-trip preservation of regression coefficients, plot generation to disk, all-failure NaN handling, default-grid sanity check, integration with the real ArmTaskEnv contract, zero-variance per-condition statistics, single-condition NaN-slope protection, and re-export via the package __init__. All fourteen tests pass.

6. Step 4 — Two-Thirds Power Law Validation Harness

The two-thirds Power Law (Lacquaniti, Terzuolo, and Viviani, 1983) is a second canonical empirical regularity of human arm motion. It states that during continuous tracing of planar curves, the tangential speed V of the hand and the curvature C of the path satisfy V = K times C raised to the power negative one third, where K is a velocity gain factor approximately constant within a single movement segment. The slope of negative one third in log-log space is the empirical fingerprint of natural human motion: humans produce slower motion through tighter curves and faster motion through straighter segments, with this specific exponent. Fischer et al. (2021) reported a Pearson correlation coefficient of R = 0.84 between log V and log C for their trained policy, demonstrating that an RL agent under appropriate reward and curriculum reproduces this property.

6.1 Module Layout

The new module src/rl_armMotion/two_d/validation/power_law.py exposes three public types: PowerLawTrial (per-trial trajectory data including the per-step end-effector positions), PowerLawResult (concatenated log V and log C samples, regression coefficients, filter thresholds, JSON serialisation, and matplotlib plotting), and PowerLawValidator (the trial driver). All three types are re-exported via the rl_armMotion.two_d.validation package.

6.2 Numerical Method

The validator runs a configurable number of independent goal-reaching trials (default thirty), logs per-step end-effector positions throughout each trial, then computes the tangential velocity V and the curvature magnitude C at every interior frame using central differences. For position samples (x_t, y_t) at uniform time spacing dt, the formulae are v_x = (x_{t+1} - x_{t-1}) / (2 dt), a_x = (x_{t+1} - 2 x_t + x_{t-1}) / dt-squared, V = square-root of (v_x squared plus v_y squared), and C = absolute-value of (v_x a_y minus v_y a_x) divided by V cubed. Central differences are computed independently within each trial so that trial boundaries do not pollute the derivative estimates.

6.3 Filtering

Two threshold filters are applied to the per-frame samples before regression: a velocity threshold (default 0.01 metres per second) below which V is too close to zero to give a meaningful log V, and a curvature threshold (default 0.10 inverse metres) below which the curve is so close to straight that the formula is unstable and the law does not meaningfully apply. Samples failing either filter are dropped, with the count of dropped samples reported in the result for diagnostic purposes. The filtered samples are then fit by closed-form ordinary least squares to the model log V = intercept + slope times log C, and both the Pearson correlation coefficient R and the coefficient of determination R-squared are reported.

6.4 The Synthetic Ellipse Benchmark

The strongest correctness check of the harness is the synthetic-ellipse smoke test. An ellipse traced at uniform parameter speed has the analytic property that V cubed times C equals a times b, the product of the ellipse semi-axes, which is a constant. This is because the curvature of the ellipse at parameter angle t is exactly a times b divided by the cube of the local speed, which makes V cubed times C identically equal to ab. Therefore log V equals log of the cube root of ab minus one third times log C, giving exactly slope equals negative one third and R-squared equals one. The synthetic-ellipse test in the smoke suite passes a sequence of one thousand positions parameterised on a circle (a = 1.0, b = 0.5) through the validator's _extract_log_v_log_c and _fit_regression methods, and verifies that the recovered slope is negative 0.33333 to seven decimal places and R-squared is 1.00000 to seven decimal places. This is the analytic answer; the harness reproduces it exactly.

6.5 Verification

Thirteen dedicated smoke tests cover validator type checks, full input validation across seven parameter classes, the synthetic-ellipse benchmark already described, NaN handling for single-sample and zero-variance regression inputs, the velocity and curvature threshold filters, integration with the real ArmTaskEnv (including a slope/R-squared computation on a random-policy trajectory), JSON round-trip, plot generation to disk, package re-export, and the regression_summary text format. All thirteen tests pass.

7. Step 5 — Hill-Type Muscle Dynamics and Seven-DOF Preset

The final integration step delivered the actuation layer that distinguishes Fischer's biomechanical setup from a standard kinematic or torque-driven arm. The Hill-type muscle model, originally introduced by A.V. Hill (1938, Proc. R. Soc. Lond. B 126:136) and elaborated by Zajac (1989, CRC Crit. Rev. Biomed. Eng. 17:359) into the modern lumped-parameter form, is the de facto standard for converting a neural-activation command into a joint-level torque in OpenSim, MuJoCo, MyoSuite, AnyBody, and SCONE. Fischer's MuJoCo musculoskeletal arm uses the same equations. This step delivers a pure-Python implementation of the same mathematics, suitable for use either as a standalone library or as a torque-actuator wrapper on the existing two-degree-of-freedom kinematic controller. It also delivers a seven-degree-of-freedom arm preset matching the kinematic structure of Fischer's arm; the preset is data only and is not yet wired into the training environment.

7.1 The Hill-Type Force Equation

The contractile-element force is computed as the product of four factors: the activation level a in the closed interval zero to one, the maximum isometric force F-max in newtons, the dimensionless force-length factor f-L of the normalised fibre length, and the dimensionless force-velocity factor f-V of the normalised fibre velocity. The activation is the input from the controller and is equivalent to the agent's normalised neural drive. F-max is a fixed parameter of the muscle and determines its absolute strength. The force-length factor peaks at the optimal fibre length L-opt, drops symmetrically for shorter and longer fibres, and is implemented in this work as the Gaussian approximation of Thelen (2003, J. Biomech. Eng. 125:70) with the half-width parameter sigma-L set to the standard value 0.45. The force-velocity factor is monotonically decreasing in the shortening (concentric) direction, equal to one at zero velocity (isometric), zero at the maximum shortening velocity v-max (unloaded shortening), and asymptotes to a plateau f-ecc-max in the lengthening (eccentric) direction. The eccentric branch is implemented as an exponential approach to the plateau matched to the slope of the concentric branch at v equals zero for smoothness.

7.2 Default Parameters

The default MuscleParameters values follow the canonical literature: F-max = 100 N, L-opt = 0.10 metres, v-max = 10 fibre lengths per second, sigma-L = 0.45, f-ecc-max = 1.5 (so eccentric force can reach one and a half times the isometric maximum), and the activation range is zero to one. These values are consistent with the values reported for upper-extremity muscles in Zajac (1989) and Murray, Buchanan, and Delp (1995, J. Biomech. 28:513). The MuscleParameters dataclass validates all eight constructor inputs in __post_init__; thirteen distinct invalid configurations are rejected with ValueError, including non-positive F-max, optimal length, v-max, or sigma-L; f-ecc-max less than one (which would give eccentric force below isometric, contradicting the physiology); and inverted activation bounds.

7.3 Integration with ArmController

ArmController in src/rl_armMotion/two_d/utils/arm_kinematics.py was extended with a new method apply_muscle_activation(joint_id, activation, muscle, moment_arm=0.05, inertia_override=None) which routes the existing kinematic controller through the Hill-type muscle model. The method maps the joint kinematic state to the muscle fibre state by treating the fibre length as L-opt plus the moment arm times the joint angle and the fibre velocity as the moment arm times the joint angular velocity divided by L-opt. It then computes the muscle force, multiplies by the moment arm to get a joint torque, and integrates one explicit Euler step at the configured dt against the joint inertia to produce an angular increment, which is then applied via the existing increment_joint method. The default moment arm of five centimetres is a reasonable order of magnitude for upper-extremity flexor muscles per Murray et al. (1995). This is a pure addition; existing call sites of ArmController are unaffected.

7.4 The Seven-DOF Fischer Arm Preset

A new preset named 7dof_fischer was added to the ArmConfiguration.get_preset registry, matching the kinematic structure of the human upper extremity that Fischer et al. (2021) implemented in MuJoCo. The seven joints are: shoulder flexion-extension (sagittal plane), shoulder abduction-adduction (frontal plane), shoulder internal-external rotation along the humeral long axis, elbow flexion-extension, forearm pronation-supination, wrist flexion-extension, and wrist radial-ulnar deviation. Segment lengths follow the standard cadaveric proportions of Winter (2009), Biomechanics and Motor Control of Human Movement, fourth edition, with the humerus thirty centimetres, the combined ulna and radius twenty-seven centimetres, and the combined hand and wrist eighteen centimetres. Joint-limit ranges follow typical anatomical values (for example, the elbow allows zero to one hundred and fifty degrees, the shoulder rotation allows minus ninety to plus ninety degrees). The total flexor-chain reach is seventy-five centimetres, which lies within the physiological range of sixty to ninety-five centimetres reported in the literature for the average adult upper extremity.

This preset is data only as of the present commit. Wiring it into the existing 2-DOF training environment as the active arm geometry would require re-deriving the observation space dimensions (which currently include 2-DOF-specific elements) and re-validating the reward function under a higher-dimensional action space, and is documented as deferred work in Section 10 of this report.

7.5 Verification

Twenty dedicated smoke tests cover (1) parameter validation across thirteen invalid configurations, (2) default values match the documented biomechanical literature, (3) the force-length factor peaks exactly at L-opt and falls off symmetrically at L = 0.07 and L = 0.13, (4) the force-velocity factor is exactly 1.0 at zero velocity (isometric), (5) the force-velocity factor is exactly zero at v-max (unloaded shortening), (6) the eccentric force-velocity factor exceeds the isometric value at slow lengthening and asymptotes correctly to f-ecc-max = 1.5 at large lengthening speeds, (7) activation is clipped to the closed interval zero to one, (8) the combined force at L-opt and v = 0 with activation 0.6 returns exactly 60 newtons, (9) the vectorised force computation agrees element-wise with the scalar version, (10) the package re-export works, (11) the seven-DOF preset is well-formed with all seven entries in each of the link lengths, masses, inertias, and joint limits, and with each joint's minimum strictly less than its maximum, (12) the preset appears in list_presets, (13) ArmController.apply_muscle_activation produces a non-zero velocity for a non-zero activation, (14) zero activation produces zero force and therefore zero velocity change, (15) the repr of HillTypeMuscle is human-readable, (16) invalid joint indices raise IndexError, (17) non-positive inertia overrides raise ValueError, (18) the force-length factor is symmetric around L-opt to floating-point precision, (19) force is exactly linear in activation (the ratio F(0.75)/F(0.25) equals 3.000 to floating-point precision), and (20) the seven-DOF flexor-chain length lies within the physiological range. All twenty tests pass. The full pre-existing test suite (forty-nine tests on the save-point commit) continues to pass without regression.

8. Unified GUI Launcher

After completion of the five Fischer-methodology steps, a small launcher window was added to give the user a single entry point for both graphical front ends of the project. This is not a Fischer-paper requirement but is a quality-of-life addition reflecting that the project now has two distinct interactive front ends (the arm-control GUI for visualisation and teleoperation, and the training GUI for live RL monitoring) and the user previously had to remember two distinct command-line invocations with separate argument syntaxes.

8.1 Implementation

The new module src/rl_armMotion/two_d/gui/__main__.py defines a small Tk window with two labelled sections. Section one is for the interactive arm-control GUI and exposes a single button. Section two is for the training GUI and exposes a dropdown for the algorithm (defaulting to SAC, with PPO and A2C available), a text field for the timestep budget (defaulting to 100,000 steps), and a text field for the save directory (defaulting to ./project_assets/outputs/fischer_session) with a Browse button that opens a directory selection dialog. Each button spawns the corresponding GUI as an independent subprocess of the same Python interpreter using subprocess.Popen with the active environment inherited; this isolates each GUI in its own clean Tk root, which is necessary because Tkinter does not support two simultaneous Tk() roots in the same process. A status bar at the bottom reports the most recent launch and the count of live subprocesses. Closing the launcher does not terminate child processes, so a user can quit the launcher while a training run continues.

8.2 Form Validation

The launcher validates inputs before spawning the training subprocess. An invalid timesteps string (not parseable as int) or a non-positive value is reported in the status bar with a friendly message rather than crashing the launcher. An unknown algorithm name is similarly rejected. The save directory is created on demand if it does not exist.

8.3 Documentation

The README.md was updated in a follow-on commit (5c4b822) to document the launcher. Section 2 of the README now contains a "Unified Launcher (recommended)" subsection. Section 4 was restructured so that the launcher command appears first as the recommended entry point, with the existing direct-launch commands preserved as alternatives 4.2 and 4.3. The training-GUI direct-launch example was updated to use --algorithm SAC and the fischer_session save directory to match the new defaults.

9. Verification, Testing, and Regression Safety

The integration was carried out under a strict regression-safety regime. Every commit was preceded by static syntax compilation of the modified files (via python -m py_compile), followed by a dedicated smoke-test suite for the new module, followed by re-running the pre-existing forty-nine-test regression suite to verify no behaviour was broken. After the final commit, an end-to-end integration test was run that exercises every Fischer component in sequence: a brief SAC training run with the curriculum and motor-babbling active, followed by a Fitts' Law sweep against the resulting model, followed by a two-thirds Power Law sweep against the same model. The pipeline completed cleanly and produced a populated curriculum-state dictionary, two well-formed regression results, and no exceptions.

9.1 Smoke Test Inventory

Step	Module	Smoke tests
1	ArmTaskEnv tolerance configuration + SAC default	7
2	AdaptiveCurriculumCallback + motor babbling	12
3	Fitts' Law harness (FittsLawValidator)	14
4	Two-thirds Power Law harness (PowerLawValidator)	13
5	Hill-type muscle model + 7-DOF preset	20
8	Unified launcher (form validation + spawn cmd)	8
	TOTAL	74

Table 2. *Smoke-test counts per Fischer integration step.*

9.2 Regression Suite Status

The pre-existing regression suite in project_assets/tests contains fifty tests across six test modules: test_data, test_gui_components, test_models, test_parallel_env, test_task_env, and test_visualization. On the save-point commit (f7f90e5), forty-nine of these tests pass and one test (test_task_env.py::TestArmTaskEnv::test_joint_limits_enforced) fails due to a pre-existing float32-versus-float64 unit-in-the-last-place precision issue in the test itself (it compares a float32 representation of 2.094 against the float64 representation, which differs by approximately 5e-8). This failure is documented as pre-existing and is not caused by the Fischer-integration work; it is present on the save-point commit. After the Fischer integration, the same forty-nine tests continue to pass; the same one test continues to fail in the same way. No regressions were introduced.

9.3 Mathematical Correctness

Two of the new modules admit exact analytical correctness checks. For the two-thirds Power Law harness, an ellipse traced at uniform parameter speed analytically satisfies V cubed times C equals the constant ab, so the regression on its log-log derivatives must give slope equals negative one third and R-squared equals one. The harness reproduces this answer to seven decimal places. For the Hill-type muscle model, the force F at the optimal fibre length and zero velocity is by construction exactly equal to the activation times F-max times one times one; the harness returns exactly 60 N for activation = 0.6 and F-max = 100 N. These are the strongest possible correctness checks because the expected answer is analytic.

10. Scope Notes — What Was Not Done

Three components of the full Fischer setup are explicitly out of scope for this implementation effort. Each is described below together with the reason it was deferred and the work that would be required to complete it.

10.1 Wiring the Seven-DOF Arm into the Training Environment

The seven-DOF Fischer arm preset added in Step 5 is data only. The ArmTaskEnv class is currently hard-locked to two degrees of freedom: line 61 of src/rl_armMotion/two_d/environments/task_env.py contains the assertion self.config.dof == 2, the action space is constructed with shape (2,), and the eleven-element observation space includes 2-DOF-specific entries. Generalising the environment to N degrees of freedom requires re-deriving the observation space (the angle-encoding, velocity, and task-error elements scale with the DOF count), re-checking the reward function for hidden 2-DOF assumptions, and updating the joint-constraint enforcement to use vector-valued joint_limits. The Fitts' Law and Power Law validators would also need their workspace-frame computations re-checked for higher-dimensional end-effector positions. This is a careful refactor on the order of one full day of focused work, and it was deferred to keep this implementation effort additive and not to risk breaking any of the existing 2-DOF tests.

10.2 MuJoCo Musculoskeletal Model File

Fischer et al. (2021) ran their experiments inside the MuJoCo physics engine using a pre-existing musculoskeletal model file in MuJoCo's XML format, the same file format that ships with the MyoSuite library. Constructing such a file from scratch requires knowledge of the MuJoCo geometric, joint, and tendon definition syntax, an anatomically correct specification of the bone, joint, and muscle attachment geometry, and validation against published kinematic and dynamic data. This is a multi-week effort and is the primary reason the full Fischer setup is not feasible inside a single CP493 directed-research project. The Hill-type muscle model delivered in Step 5 is mathematically identical to the one MuJoCo applies internally, so the actuation physics is in place; what is missing is the multi-body simulation substrate.

10.3 Multi-Muscle-Per-Joint Anatomical Geometry

A fully realistic upper-extremity model has multiple muscles spanning each joint (the elbow alone has biceps brachii, brachialis, brachioradialis as flexors and triceps brachii as extensor), with each muscle having an anatomically correct line-of-action that produces a moment arm that varies with joint angle. The implementation in Step 5 uses a single muscle per joint with a constant moment arm, which is sufficient to deliver the correct force-generation law (the Hill-type equation) but does not capture joint-angle-dependent moment arms or muscle redundancy. Adding multi-muscle geometry requires anatomical data tables (such as those of Murray, Buchanan, and Delp, 1995, J. Biomech. 28:513) and a moment-arm interpolation framework, and is best done after the multi-body MuJoCo substrate is in place.

10.4 Long Training Run and Real Validation Numbers

The Fitts' Law and Power Law harnesses delivered in Steps 3 and 4 measure whether a trained policy reproduces the two empirical regularities. The harnesses are correct to the level of analytical precision (slope of negative one third recovered to seven decimal places on a synthetic ellipse), but the actual coefficients measured against any given trained model depend on the quality of training. Fischer's reported R-squared = 0.9986 and R = 0.84 emerged only after their multi-million-step training run on a multi-DOF musculoskeletal arm. A demonstration training run of one hundred thousand steps on the two-degree-of-freedom arm in this project will not approach those numbers; a longer run (approximately one million steps) on the existing 2-DOF setup is the natural next step toward producing realistic validation values, and this run can be initiated at any time from the unified GUI launcher described in Section 8.

11. How to Use the New Capabilities

11.1 Launching the GUIs

The simplest way to interact with all the new functionality is via the unified launcher. After activating the project virtual environment, run:

```
cd /Users/ranjotsandhu/Documents/Project
source venv/bin/activate
python -m rl_armMotion.two_d.gui
```

A small window titled "RL Arm Motion - Launcher" opens. The arm-control GUI is launched by clicking the Open Arm Control GUI button. The training GUI is launched by setting the algorithm (defaulting to SAC), the timestep budget (defaulting to 100,000), and the save directory (defaulting to ./project_assets/outputs/fischer_session), then clicking the Start Training GUI button. The training subprocess will print the curriculum announcement and proceed through the five-thousand-step motor-babbling phase before SAC begins gradient updates.

11.2 Running the Validators on a Trained Model

After a model has been trained and saved, the two validators can be run on it as follows:

```
from rl_armMotion.two_d.environments.task_env import ArmTaskEnv
from rl_armMotion.two_d.validation import FittsLawValidator,
PowerLawValidator
from stable_baselines3 import SAC

env = ArmTaskEnv()
model = SAC.load(
"./project_assets/outputs/fischer_session/sac_model.zip",
env=env,
)

fl = FittsLawValidator(model=model, env=env)
fl_result = fl.run(n_trials_per_condition=15, seed=42)
print(fl_result.regression_summary())
fl_result.save_json("./fitts_law_result.json")
fl_result.plot(save_path="./fitts_law_plot.png")

pl = PowerLawValidator(model=model, env=env)
pl_result = pl.run(n_trials=30, seed=42)
print(pl_result.regression_summary())
pl_result.save_json("./power_law_result.json")
pl_result.plot(save_path="./power_law_plot.png")
```

11.3 Using the Hill-Type Muscle Model

The Hill-type muscle is exposed at the package level. It can be used standalone for biomechanical investigation:

```
from rl_armMotion.two_d.utils import HillTypeMuscle, MuscleParameters
```

```
m = HillTypeMuscle() # Fischer-aligned defaults
print(m)
print("f_L peaks at L_opt:", m.force_length(0.10)) # 1.0
print("f_V isometric:", m.force_velocity(0.0)) # 1.0
print("f_V eccentric:", m.force_velocity(-100.0)) # 1.5
print("force at full activation:", m.force(1.0, 0.10, 0.0)) # 100 N
```

It can also be wired to drive a single joint of the existing two-degree-of-freedom arm via the new ArmController.apply_muscle_activation method:

```
from rl_armMotion.two_d.config import ArmConfiguration
from rl_armMotion.two_d.utils.arm_kinematics import ArmController

cfg = ArmConfiguration.get_preset("2dof_simple")
ctrl = ArmController(cfg)
muscle = HillTypeMuscle()
for _ in range(10):
ctrl.apply_muscle_activation(joint_id=0, activation=0.8, muscle=muscle)
print("Final shoulder velocity:", ctrl.velocities[0])
```

11.4 Inspecting the 7-DOF Arm Preset

```
from rl_armMotion.two_d.config import ArmConfiguration

cfg7 = ArmConfiguration.get_preset("7dof_fischer")
print("DOF:", cfg7.dof)
print("Total reach (m):", sum(cfg7.link_lengths))
print("Joint limits (rad):", cfg7.joint_limits_min, cfg7.joint_limits_max)
```

12. Summary of Implementation Statistics

The complete Fischer-integration effort consists of six commits totalling approximately 2,200 inserted lines of Python source code, distributed as shown in Table 3.

Commit	Description	Files	Lines added
77b7180	Step 1: SAC default + configurable goal tolerance	3	92
9d31252	Step 2: adaptive curriculum + motor babbling	3	307
6f995f0	Step 3: Fitts' Law validation harness	3	695
5e6b467	Step 4: 2/3 Power Law validation harness	2	622
7431988	Unified GUI launcher	1	291
5c4b822	README documentation update for the launcher	1	32
5f37a26	Step 5: Hill-type muscle dynamics + 7-DOF preset	4	474
	TOTAL	17	2,513

Table 3. *Per-commit line and file counts for the Fischer integration effort. The file counts include the same file modified across multiple commits (e.g., task_env.py modified in Steps 1, 3, and 5).*

12.1 Component Coverage

Pillar	Deliverable	Lines	Tests
1	SAC default + configurable tolerance	~92	7
2	AdaptiveCurriculumCallback + motor babbling	~307	12
3	Fitts' Law harness	~695	14
4	2/3 Power Law harness	~622	13
5	Hill-type muscle + 7-DOF preset	~474	20
	TOTAL	~2,190	66

Table 4. *Lines and dedicated smoke tests per Fischer methodological pillar.*

References

Fischer, M., Hoinville, T., Eickhoff, S. B., & Lilienthal, A. J. (2021). Reinforcement learning control of a biomechanical model of the upper extremity. *Scientific Reports*, 11, 14445. https://doi.org/10.1038/s41598-021-93760-1

Fitts, P. M. (1954). The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6), 381–391.

Hill, A. V. (1938). The heat of shortening and the dynamic constants of muscle. *Proceedings of the Royal Society of London. Series B, Biological Sciences*, 126(843), 136–195.

Lacquaniti, F., Terzuolo, C., & Viviani, P. (1983). The law relating the kinematic and figural aspects of drawing movements. *Acta Psychologica*, 54(1–3), 115–130.

Murray, W. M., Buchanan, T. S., & Delp, S. L. (1995). The isometric functional capacity of muscles that cross the elbow. *Journal of Biomechanics*, 28(5), 513–525.

Thelen, D. G. (2003). Adjustment of muscle mechanics model parameters to simulate dynamic contractions in older adults. *Journal of Biomechanical Engineering*, 125(1), 70–77.

Viviani, P., & Stucchi, N. (1992). Biological movements look uniform: Evidence of motor-perceptual interactions. *Journal of Experimental Psychology: Human Perception and Performance*, 18(3), 603–623.

Winter, D. A. (2009). *Biomechanics and Motor Control of Human Movement* (4th ed.). John Wiley & Sons.

Zajac, F. E. (1989). Muscle and tendon: Properties, models, scaling, and application to biomechanics and motor control. *Critical Reviews in Biomedical Engineering*, 17(4), 359–411.

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80:1861–1870.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization algorithms. *arXiv preprint* arXiv:1707.06347.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8.

Towers, M. et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint* arXiv:2407.17032.

Appendix A — Commit Log

The complete sequence of commits constituting the Fischer integration, in chronological order from the save point to the present head of the claude/fischer-integration branch, is shown in Table 5.

Hash	Date	Step	Subject
f7f90e5	2026-05-05	save point	Save point: documentation, references, training results
77b7180	2026-05-05	step 1	SAC as default algorithm and configurable goal tolerance
9d31252	2026-05-05	step 2	Adaptive curriculum and motor babbling
6f995f0	2026-05-06	step 3	Fitts' Law validation harness
5e6b467	2026-05-06	step 4	Two-thirds Power Law validation harness
7431988	2026-05-06	launcher	Add unified GUI launcher: python -m rl_armMotion.two_d.gui
5c4b822	2026-05-06	launcher	Document the unified GUI launcher in the README
5f37a26	2026-05-06	step 5	Hill-type muscle dynamics + 7-DOF preset

Table 5. *Chronological commit log of the Fischer integration effort.*

All commits were made on the dedicated branch claude/fischer-integration, which was branched from main at the save point f7f90e5. The save point itself is preserved as the immutable git tag save-point-2026-05-05 and as the named branch backup/save-point-2026-05-05, either of which can be checked out to restore the pre-Fischer project state.

Appendix B — File Inventory

The Fischer integration touches seventeen files across the project, of which five are new modules and the remainder are targeted modifications to existing modules. Table 6 lists each file with its role.

File path	Role	Step
src/rl_armMotion/two_d/training/curriculum_callback.py	NEW	2
src/rl_armMotion/two_d/validation/__init__.py	NEW	3
src/rl_armMotion/two_d/validation/fitts_law.py	NEW	3
src/rl_armMotion/two_d/validation/power_law.py	NEW	4
src/rl_armMotion/two_d/utils/muscle_model.py	NEW	5
src/rl_armMotion/two_d/gui/__main__.py	NEW	launcher
src/rl_armMotion/two_d/environments/task_env.py	modified	1, 3
src/rl_armMotion/two_d/gui/training_gui.py	modified	1
src/rl_armMotion/two_d/models/agents/sac_agent.py	modified	1, 2
src/rl_armMotion/two_d/training/ppo_trainer_wrapper.py	modified	2
src/rl_armMotion/two_d/utils/arm_kinematics.py	modified	5
src/rl_armMotion/two_d/utils/__init__.py	modified	5
src/rl_armMotion/two_d/config/arm_config.py	modified	5
README.md	modified	launcher

Table 6. *All files modified or added by the Fischer integration.*